



Bootcamp

Inter IIT Tech Meet 15.0

Clue Chain Hunt

Robotics Club

Team Size : Individual

Submission Deadline : [DEADLINE]





Problem Statement

Robots on real missions have to read their surroundings, reason about what they see and act on it, all without a human in the loop. Combining computer vision with ROS 2 navigation is what makes this possible.

This challenge builds on the workshop exercises on OpenCV, ROS 2 and Gazebo. A **leader robot** must solve a chain of visual clues hidden in an unknown arena to reach a treasure. Each clue board carries an ArUco marker and a QR code that points to the next board. A second, **camera-only follower robot** must keep up with the leader throughout the hunt. The robots, arena and simulation are provided; your task is the **autonomy**.

Objectives:

1. Clue Detection (OpenCV) :

Detect the ArUco marker (DICT_4X4_50, 0.24 m) on each clue board and decode the QR code beside it.

2. Board Localisation (TF2) :

Estimate each board's pose with solvePnP, transform it into the map frame, and publish it.

3. Mapping & Navigation :

Build a 2D map of the arena with slam_toolbox, save it, and navigate on it with Nav2 (AMCL).

4. Autonomous Clue Hunt :

From a single launch command, follow the chain from board 1 to the treasure with no human input. Interpret every clue type and reject decoy and look-alike boards.

5. Follower Robot :

Using only its camera and wheel odometry, the follower must stay 0.6 — 2.0 m behind the leader by tracking the ArUco tag on the leader's back. It must not hit any wall or the leader.

6. Optional (Bonus) :

Run the hunt without a pre-built map (SLAM running live), and visualise the boards, pillars and the follower's path in RViz2.



Problem Statement

The Clue Boards:

Each board shows an **ArUco marker** (left) and a **QR code** (right). The QR code contains one line:

```
HUNT : <id> : <token> : [TREASURE] <command>
example: HUNT:3:BCB8:BETWEEN BLUE GREEN 0.59
```

GOTO x y	Next board is at (x, y) in the map frame (board 1 only).
PILLAR C	Next board is within 1.8 m of the red / green / blue pillar.
BETWEEN A B f	Next board is within 1.8 m of the point $A + f \cdot (B - A)$ between two pillars.
REL a b	Next board is at ArUco centre + $a \cdot X + b \cdot Y$ in the board's frame (X out of the face, Y to the reader's right).
TREASURE REL a b	Location of the treasure. Drive onto it (within 0.3 m).

Chain token: the first 4 hex characters (upper case) of the SHA-1 of the previous valid clue text (use "START" for board 1). Decoy boards and **look-alikes** (same id, wrong token) must be ignored.

The Robots:

1. Leader :

Differential drive with a 360° LiDAR, an RGB camera and an IMU. Spawns at the origin, and carries an ArUco tag (id 49, 0.12 m) on its back.

2. Follower :

Differential drive with an RGB camera and wheel odometry only (no LiDAR, no map). It may not use any of the leader's data.

Topics to Publish:

/hunt/clues	Full text of every valid clue, in order (std_msgs/String)
/hunt/boards	"id x y": estimated board position in the map frame (std_msgs/String)
/hunt/treasure	Treasure position in the map frame (geometry_msgs/PoseStamped)
/follower/cmd_vel	Velocity commands for the follower (geometry_msgs/Twist)

Full details (frames, topics, arena layout) are in the README of the starter package.





Problem Statement

Deliverables:

1. Technical Report (max 5 pages) :

- Overview of your system: nodes, topics and launch files.
- Explanation of clue detection, pose estimation and frame transforms.
- Your clue-solving and search strategy, and your follower design.
- Screenshots of the map, the hunt and the follower, plus failure cases.

2. Demo Video :

One uninterrupted autonomous run in the practice world: the leader solves the full chain and the follower follows it. Show Gazebo and RViz2.

3. Code Package :

Your `clue_hunt_solver` package with all nodes, launch files, parameter configurations and the saved map, and a README with setup instructions.

4. Optional (Bonus) :

A run without a pre-built map, and RViz2 visualisation of detections and the follower's path.

Evaluation Criteria:

Aspect	Description	Weight
Perception	ArUco + QR decoding, token validation and board position accuracy	20 %
Navigation	Map quality and reliable autonomous navigation with Nav2	15 %
Clue Hunt	Full chain and treasure on the hidden test worlds	25 %
Follower Robot	Camera-only following, distance kept, no collisions	20 %
Demonstration	Video clarity, results and overall execution	10 %
Documentation	Report quality, reproducibility and clarity	10 %
	Total	100 %

Solutions are tested on hidden worlds, where boards are moved, pillar colours are swapped and the lighting is changed. Shortlisted candidates may be asked to explain their code.





Problem Statement

Additional Recommendations

1. Software Tools :

Ubuntu 22.04, ROS 2 Humble, Gazebo Fortress, RViz2, OpenCV.

2. Packages :

nav2_bringup, slam_toolbox, nav2_simple_commander, cv_bridge, tf2_ros, ros_gz_bridge

3. Resources :

The starter package (robots, practice world, launch files, templates and README) and the **Ubuntu 22.04 + ROS 2 Humble + Gazebo installation guide** are in the [Drive folder](#).

Suggested Steps:

1. Set up the system using the installation guide and build the starter package.
2. Launch the practice world and view both cameras and the TF tree.
3. Detect board 1 and decode its QR code from the start position.
4. Map the arena with slam_toolbox, save the map and test Nav2 goals.
5. Chain it all together: search → approach → read → validate → navigate.
6. Add the follower: detect tag 49 and follow the leader's path.

Baseline Scenario:

Use the provided practice world (c1ue_hunt_gazebo).

- The leader maps the arena and saves map.yaml.
- The leader reads board 1 from its start pose and navigates to board 2.
- Extend this to the full chain, then add the follower driving behind the leader.

Contact Details

[NAME] : [PHONE]

[NAME] : [PHONE]

